

Московский физико-технический институт(государственный университет)
Факультет радиотехники и вычислительной техники
Кафедра информатики и вычислительной техники

Выпускная квалификационная работа бакалавра

Мониторинг производительности ядра ОСРВ Эльбрус с помощью Linux Trace Toolkit

Студент: Мустафин Т.Р. гр. 913

Научный руководитель: к.т.н. Ревякин В.А.

Решаемые проблемы

- Мониторинг времени отклика ядра
- Выяснение причин выбросов по времени отклика.

Возможные пути решения:

SystemTap, DTrace, LTTng

Из рассмотренных вариантов выбран LTTng

LTTng project

Linux Trace Toolkit (Пакет Трассировки Линукс) — набор программ, позволяющий отслеживать происходящие в ОС события.

Проект LTTng (LTT next generation) — приемник проекта LTT

Подсистемы LTTng :

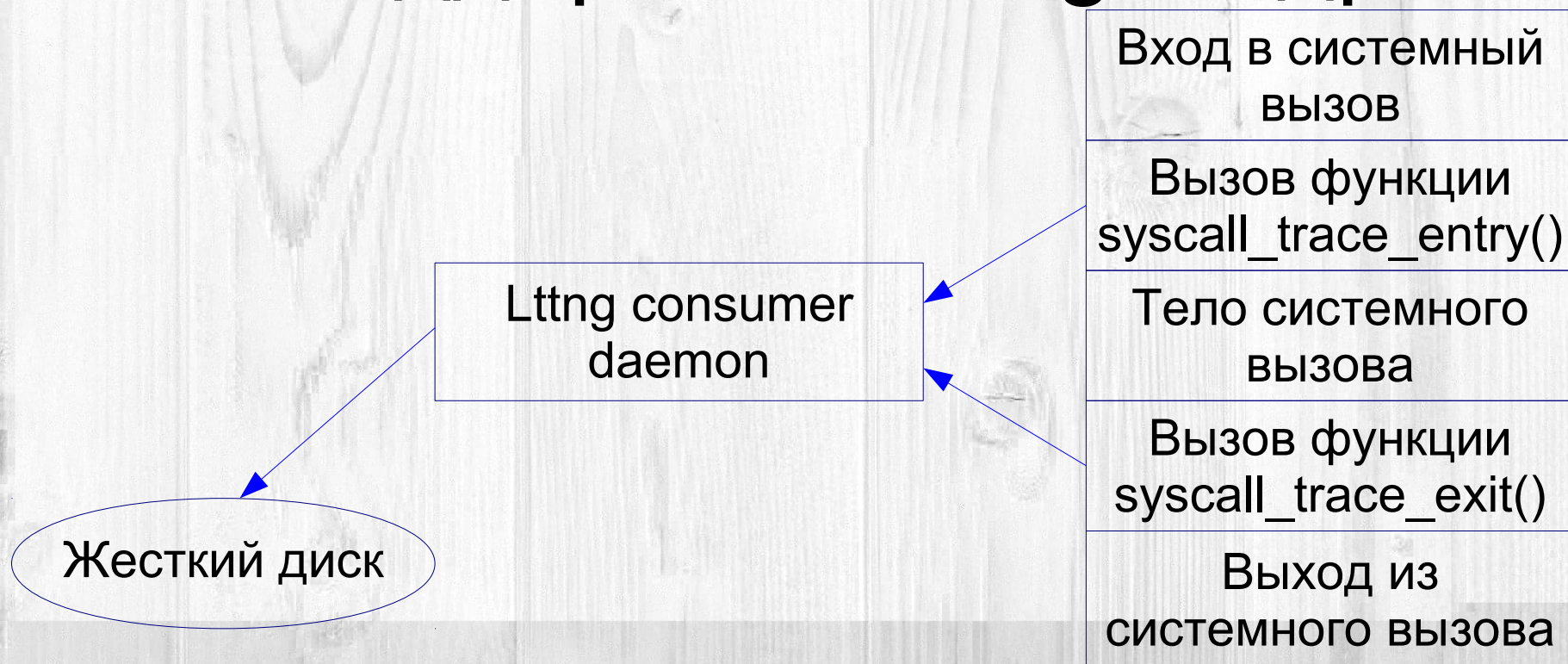
- Поддержка возможности трассировки в ядре
- Модуль управления трассировками(Control module)
- Графическая оболочка

Постановка задачи

Портировать на архитектуру Эльбрус все подсистемы проекта LTTng :

- Поддержка LTTng в ядре:
tracepoints, ltt-consumer-daemon
- Модуль управления
- Графический интерфейс

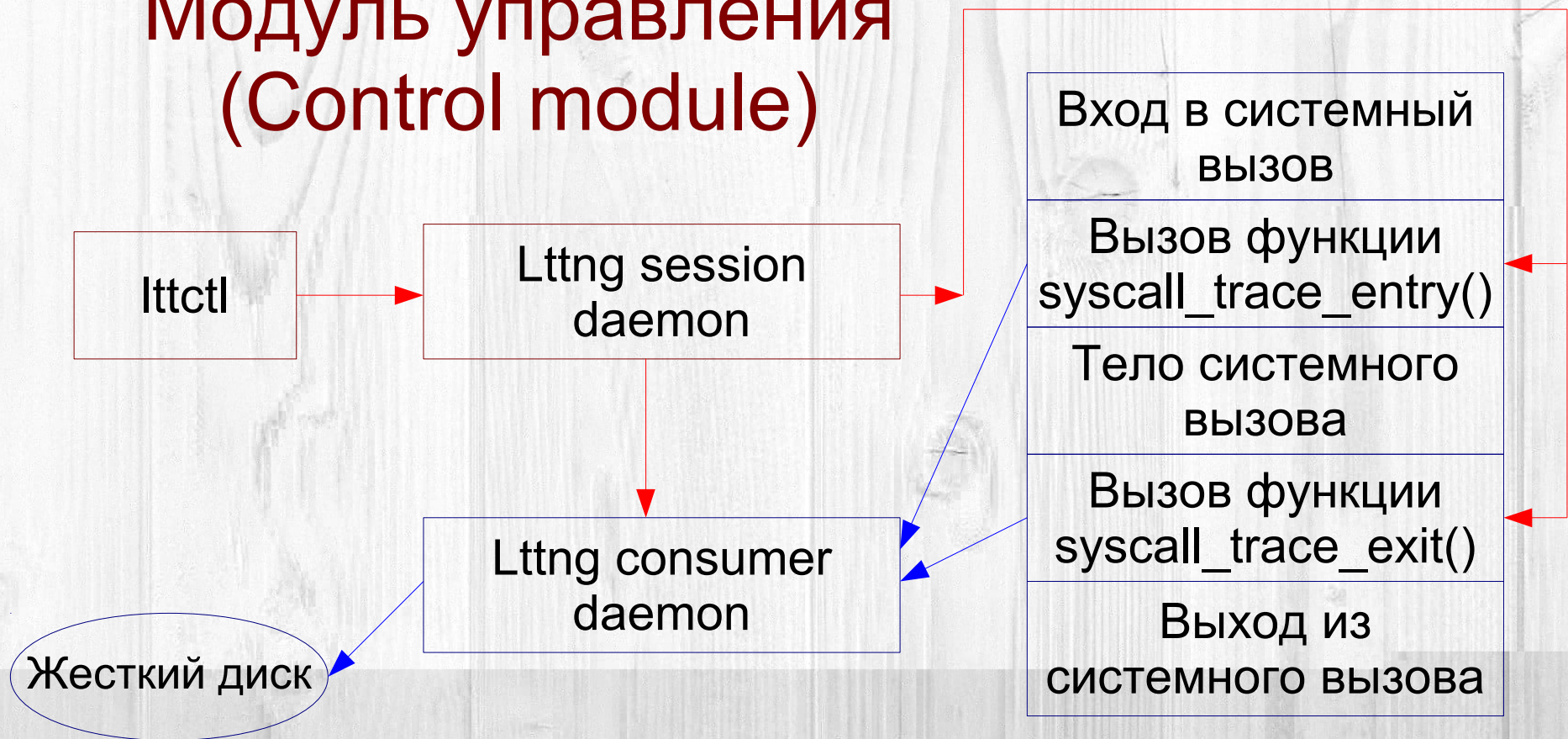
Поддержка LTTng в ядре



- Lttng consumer daemon процесс, сохраняющий собранную информацию на диск в формате ctf (common trace format)

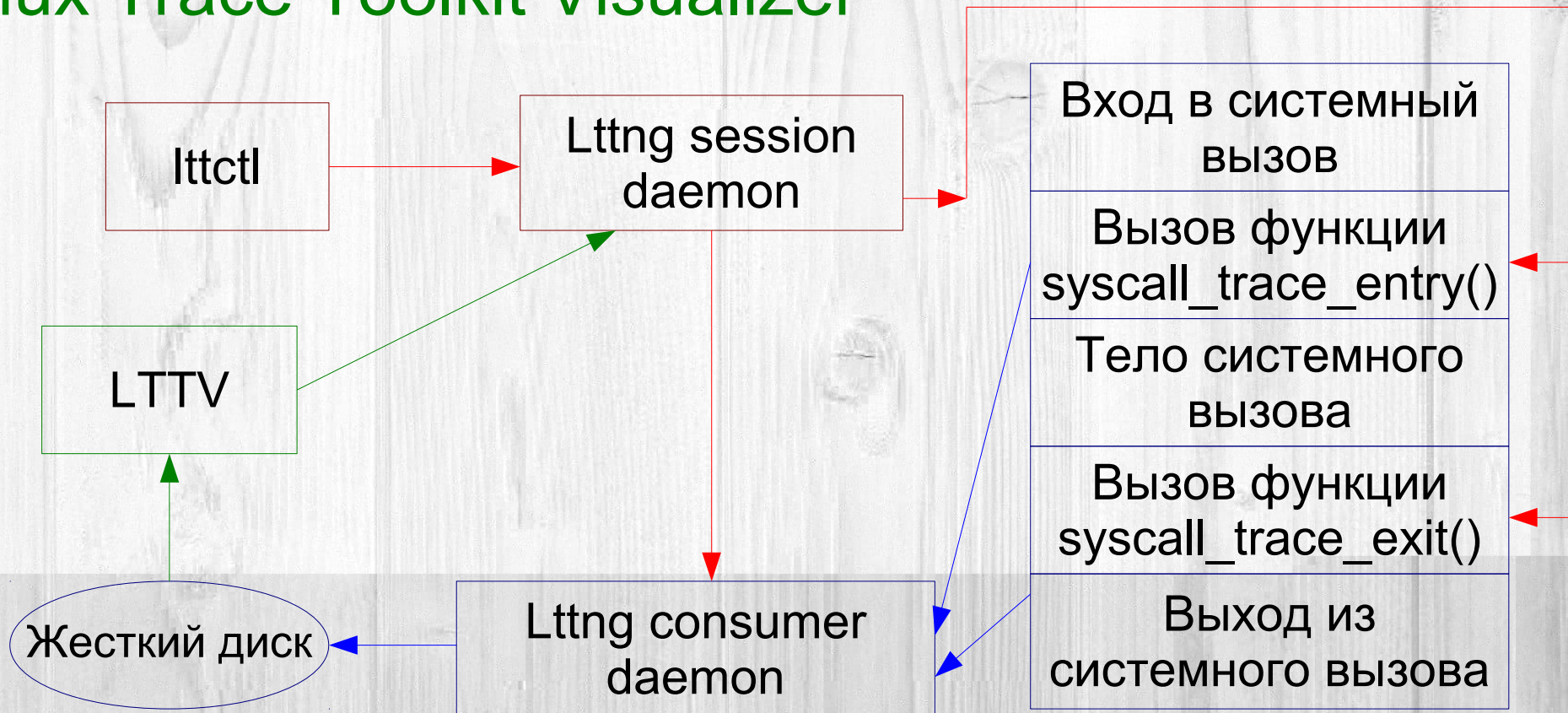
- Tracepoints метки в коде операционной системы собирающие информацию о ходе работы ОС

Модуль управления (Control module)



- **lttn session daemon** — процесс контролирующий различные сессии трассировки
- **lttn command line interface (Ittctl)** — интерфейс командной строки пользователя для управления трассировками через **lttn_session_daemon**

Графическая оболочка Linux Trace Toolkit Visualizer



- Наглядное изображение полученных результатов и их анализ
- Графический интерфейс управления трассировками

Взаимодействие подсистем

Графическая
часть

Control модуль

Ядро

Ittv —
графическая
оболочка

Ittctl — интерфейс
командной строки

tracepoints

Готовая
трасса(ctf)

Команды
управления

Команды
управления

Вкл/выкл точки
трассировки

Информация
о событии

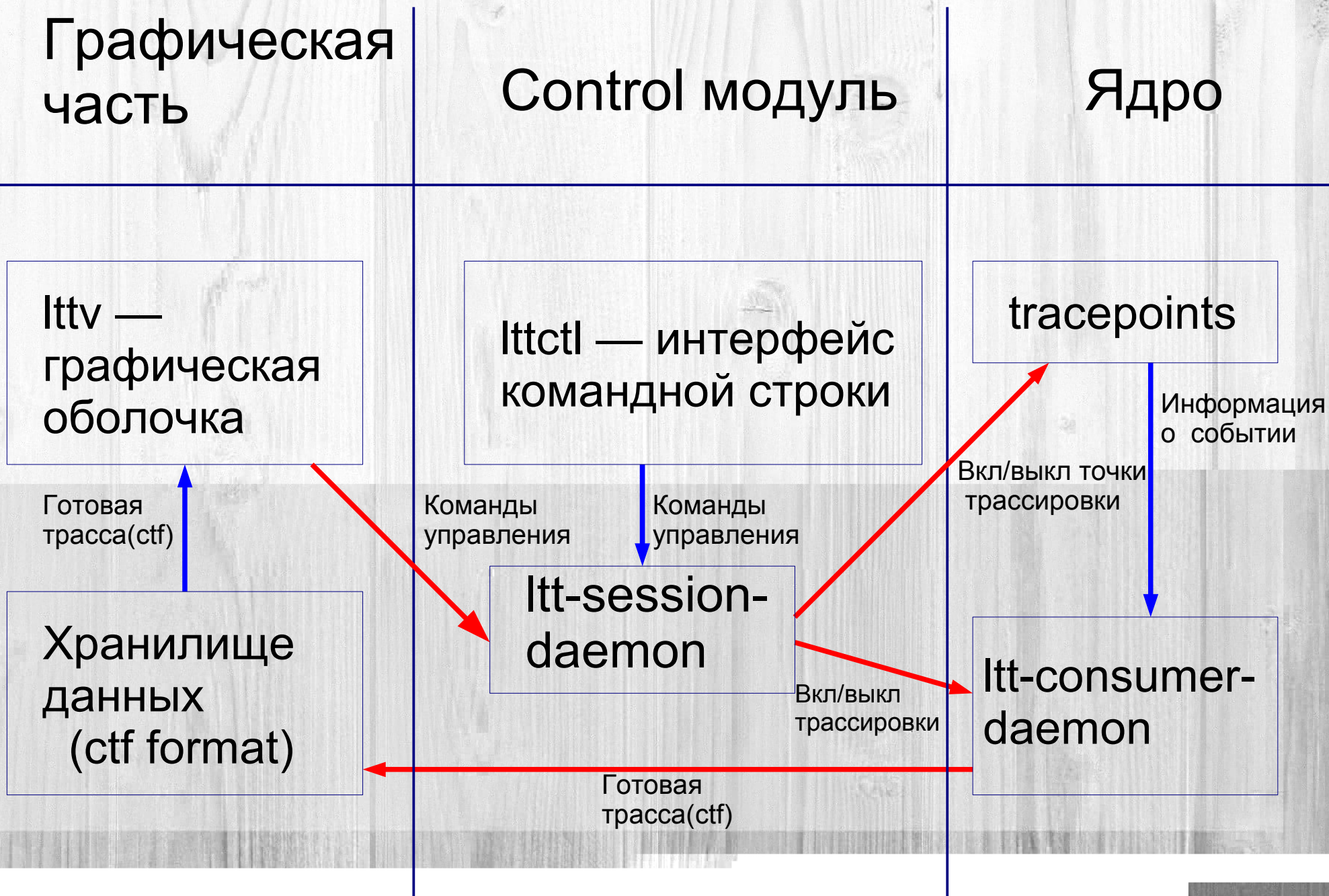
Хранилище
данных
(ctf format)

Itt-session-
daemon

Вкл/выкл
трассировки

Itt-consumer-
daemon

Готовая
трасса(ctf)



Реализация поддержки системы трассировки в ядре

Две новые функции для согласования программных интерфейсов LTTng и OCPB Эльбрус

arch/e2k/kernel/ptrace.c

Функция в начале системного вызова:

```
||+extern int syscall_trace_entry(struct pt_regs *regs);
```

Функция в конце системного вызова:

```
||+extern void syscall_trace_leave(struct pt_regs *regs);
```


Реализация поддержки системы трассировки в ядре

Модификация входов в системные вызовы:

Реализованы функции `system_call_entry()` и `system_call_leave()`, чтобы они заработали, надо внести правки в `trap_table.S`:

- `arch/e2k/kernel/trap_table.S`:

```
+             cmpandb %dr13, _TIF_SYSCALL_TRACE| _TIF_KERNEL_TRACE | \
+             _TIF_SYSCALL_TRACEPOINT, %pred15; \
```

- `arch/e2k/kernel/ thread_info.h`:

```
+#define TIF_KERNEL_TRACE      7          /* kernel trace active */
+#define _TIF_KERNEL_TRACE     (1 << TIF_KERNEL_TRACE)
```

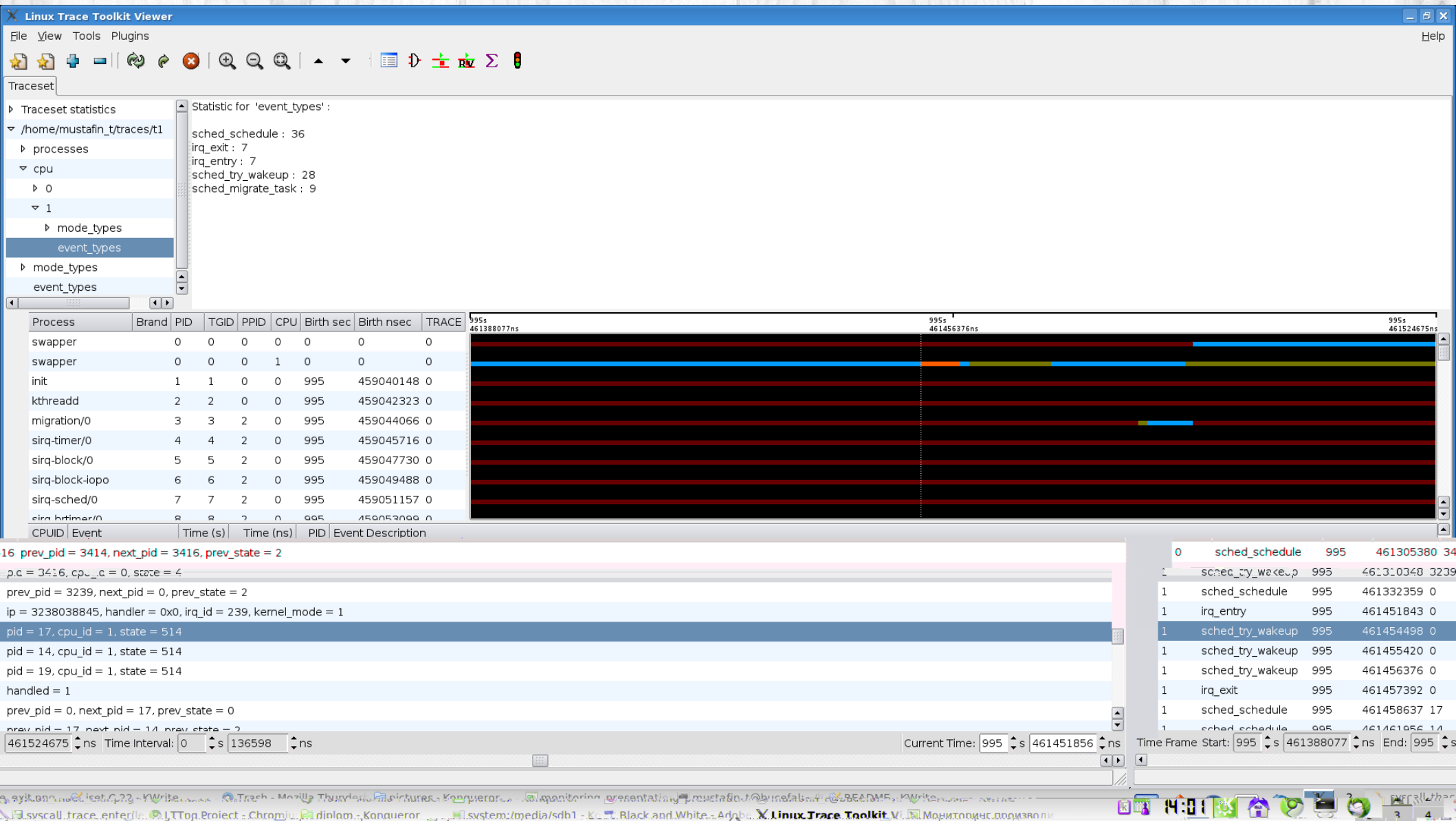

Поддержка control модуля и графической оболочки

Реализация следующих зависимостей:

- Lttctl:
 - libport 1.13 — библиотека для разбора параметров командной строки
- Lttv:
 - libgtk2.0, libgtk2.0-dev — библиотека графического пользовательского интерфейса GTK+
 - libglib2.0-0, libglib2.0-dev — низкоуровневая библиотека, расширяющая возможности, предоставляемые стандартной библиотекой libc языка C
 - libport0, libport-dev — библиотека для разбора параметров командной строки
 - librango1.0, librango1.0-dev — библиотека для отображения текста на разных языках
 - libc6, libc6-dev — стандартная библиотека языка Си

Собраны пакеты lttctl.deb и lttv.deb и загружены в локальный репозиторий os.lab.sun.mcst.ru/debian

Экспериментальные результаты



Результаты

- Исследованы алгоритмы работы и взаимодействия следующих подсистем:
 - поддержка LTTng в ядре
 - модуль управления
 - графической оболочки
- Проведена интеграция системы трассировки в ядро ОСРВ Эльбрус
- Собраны пользовательские библиотеки в рамках проекта Debian

На данный момент LTTng используется в рабочем процессе разработки ОСРВ Эльбрус

Спасибо за внимание!


```

+int syscall_trace_entry(struct pt_regs *regs)
+{
+    int ret = 0;
+    /* do the secure computing check first */
+    secure_computing(regs->sc_restart);
+
+    if (test_thread_flag(TIF_SYSCALL_TRACE))
+        ret = tracehook_report_syscall_entry(regs);
+
+    if (unlikely(test_thread_flag(TIF_SYSCALL_TRACEPOINT)))
+        trace_sys_enter(regs, regs->sc_restart);
+
+    if (unlikely(current->audit_context) && !ret)
+        audit_syscall_entry(AUDIT_ARCH_E2K,
+                            regs->arg1,
+                            regs->arg2,
+                            regs->arg3,
+                            regs->arg4,
+                            regs->arg5);
+    return ret;
+}
+
+void syscall_trace_leave(struct pt_regs *regs)
+{
+    if (unlikely(current->audit_context))
+        audit_syscall_exit(AUDITSC_RESULT(regs->sc_restart),
+                            regs->sc_restart);
+
+    if (unlikely(test_thread_flag(TIF_SYSCALL_TRACEPOINT)))
+        trace_sys_exit(regs, regs->sc_restart);
+
+    if (test_thread_flag(TIF_SYSCALL_TRACE))
+        tracehook_report_syscall_exit(regs, 0);
+}
+

```